

CS/CE 224/227: Object Oriented Programming and Design Methodologies

Week 04/Unit 04: Introduction to Object Oriented Programming

Tauqeer Saleem

Slides based on Dr Faisal Alvi's slides

For any suggested modifications please email: tauqeer.saleem@sse.habib.edu.pk



Revision



Dynamic Memory Allocation

- Recall from the last lecture that in C++ memory may be allocated dynamically.
- This dynamic memory is allocated on the heap, as opposed to memory allocated for static arrays (and primitive data types) on the stack.
- The allocation of memory may be done using the following statement:

```
int* att = new int[10];
```



Memory Leak

- This allocated dynamic memory must be explicitly deallocated after use, since it retains itself after the block of code that requested the allocation ends.
- To see this, have a look at the following program:



Memory Leak

```
#include <iostream>

int* global;
//global variable to demonstrate memory leak

void funct(void) {
    int* arr = new int[10];
    std::cout << "Inside Function funct" << "\n";
    for (int i = 0; i < 10; ++i) {
        arr[i] = i * 2;
        std::cout << "arr[" << i << "] = " << arr[i] << ", ";
    }
    std::cout << "\n";
    global = arr;
    // Memory leak: 'delete[] arr;' is missing
    // Function ends without deallocating the allocated memory
}
```



Memory Leak (contd.)

```
int main() {  
    // Allocate an array of 10 integers  
    funct();  
    std::cout << "Inside Main " << "\n";  
  
    for (int i = 0; i < 10; ++i) {  
        std::cout << "arr[" << i << "] = " << *(global + i) << ", ";  
    }  
    std::cout << "\n";  
    return 0;  
}
```



Dangling Pointers

- A pointer pointing to a memory location that has explicitly been deleted is known as a dangling pointer.
- A dangling pointer must be set to nullptr.
- In the last example, we must use

```
delete[] arr; //to delete the allocated  
memory
```

```
global = nullptr; //to set the pointer to  
null
```



Function Overloading

- We saw in our lab that function overloading is a mechanism whereby, the same functionality is implemented for various parameter types.
- A function is said to be **overloaded** if an implementation of it has the same name but different signature.
- The signature of a function is the number, type and order of its parameters.
- Note that you may get a compiler error, if a function name and signature is the same, however the return type is different.



Function Overloading (Examples)

```
#include <iostream>

// Function to print an integer
void printData(int value) {
    std::cout << "Integer: " << value << std::endl;
}

// Function to print a floating-point number
void printData(double value) {
    std::cout << "Double: " << value << std::endl;
}

// Function to print a character array (C-string)
void printData(const char* value) {
    std::cout << "Character Array: " << value << std::endl;
}
```



Function Overloading (Examples)

```
int main() {  
    int intValue = 42;  
    double doubleValue = 3.14;  
    const char* charArrayValue = "Hello, World!";  
  
    // Call the overloaded functions  
    printData(intValue);           // Calls printData(int)  
    printData(doubleValue);       // Calls printData(double)  
    printData(charArrayValue);    // Calls printData(const char*)  
  
    return 0;  
}
```



Classes and Objects

- Recall from your HW 1 that you must have used structures.
- A structure in C++ is a collection of simple variables.
- The variables in a structure can be of different types: Some can be `int`, some can be `float`, and so on.
- A class in Object Oriented Programming is a collection of
 - **data**, and
 - the **operations** on that data.
- In terms of C++ programming, a class consists of variables that may be primitive data types or classes, as well as functions (on these variables).



Classes and Objects

- A class in object oriented programming can be thought of as a “data type”.
- However this data type is more involved than primitive data types as it contains both data and functions.
- **An object is an instance of a class.**
- In terms of primitive data types, an object is like a variable that may hold some value.
- Let's see an example.



Basics of OOP

Defining Class and Object



Classes and Objects (Example)

```
class smallobj
//define a class
{
    private:
        int somedata; //class data
    public:
        void setdata(int d) //member function to set data
        { somedata = d; }

        void showdata() //member function to display data
        { cout << "Data is " << somedata << endl; }
};
```



Classes and Objects (Example)

```
int main()
{
    smallobj s1, s2; //define two objects of class smallobj
    s1.setdata(1066); //call member function to set data
    s2.setdata(1776);
    s1.showdata(); //call member function to display data
    s2.showdata();
    return 0;
}
```

Classes

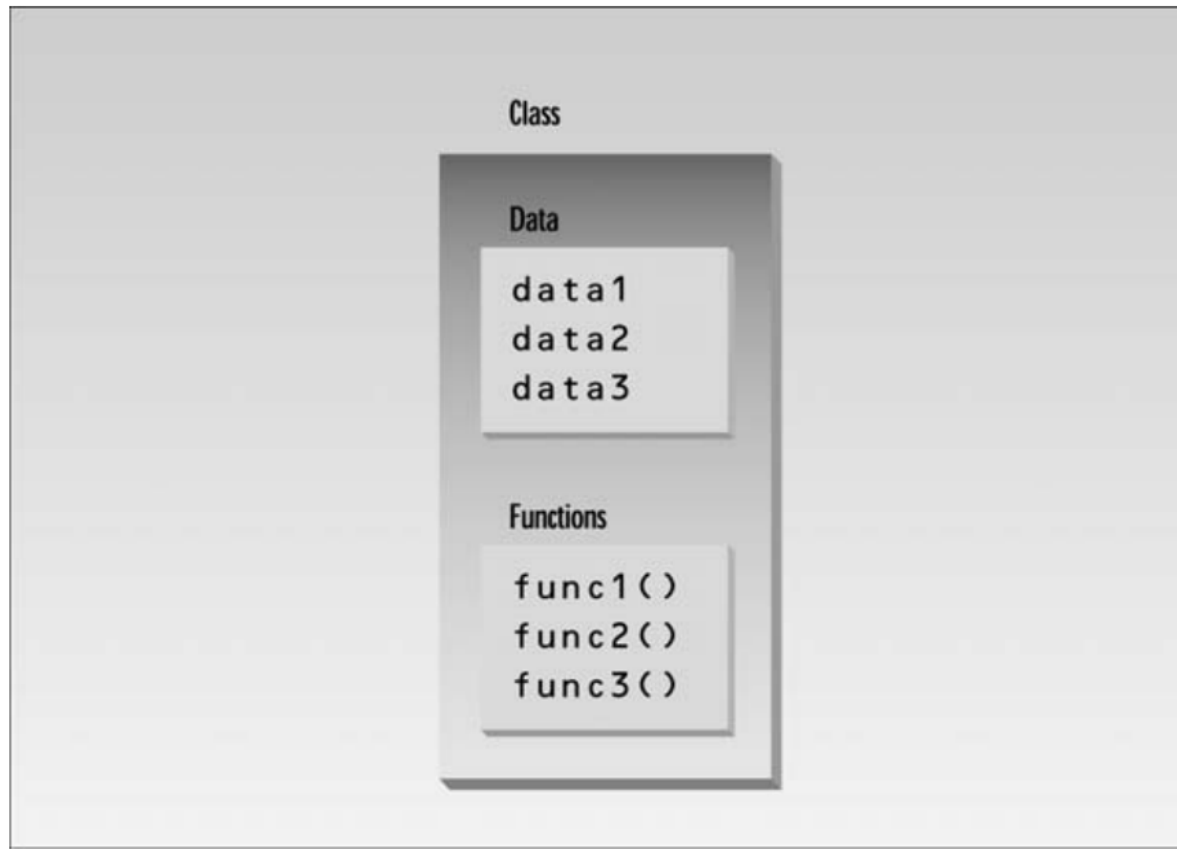


FIGURE 6.1

Classes contain data and functions.



Analysis of the Example

```
class smallobj      Class
//define a class    Definition
{
    private:
        int somedata; //class data    Private Variables
    public:
        void setdata(int d) //member function to set data
        { somedata = d; }
        void showdata() //member function to display data
        { cout << "Data is " << somedata << endl; }
};
```



Analysis of the Example

```
int main()  
{  
    smallobj s1, s2; // Object Instantiation two objects of class smallobj  
    s1.setdata(1066); //call member function to set data  
    s2.setdata(1776); // Calling functions  
    s1.showdata(); //call member function to display data  
    s2.showdata();  
    return 0;  
}
```



Class Members

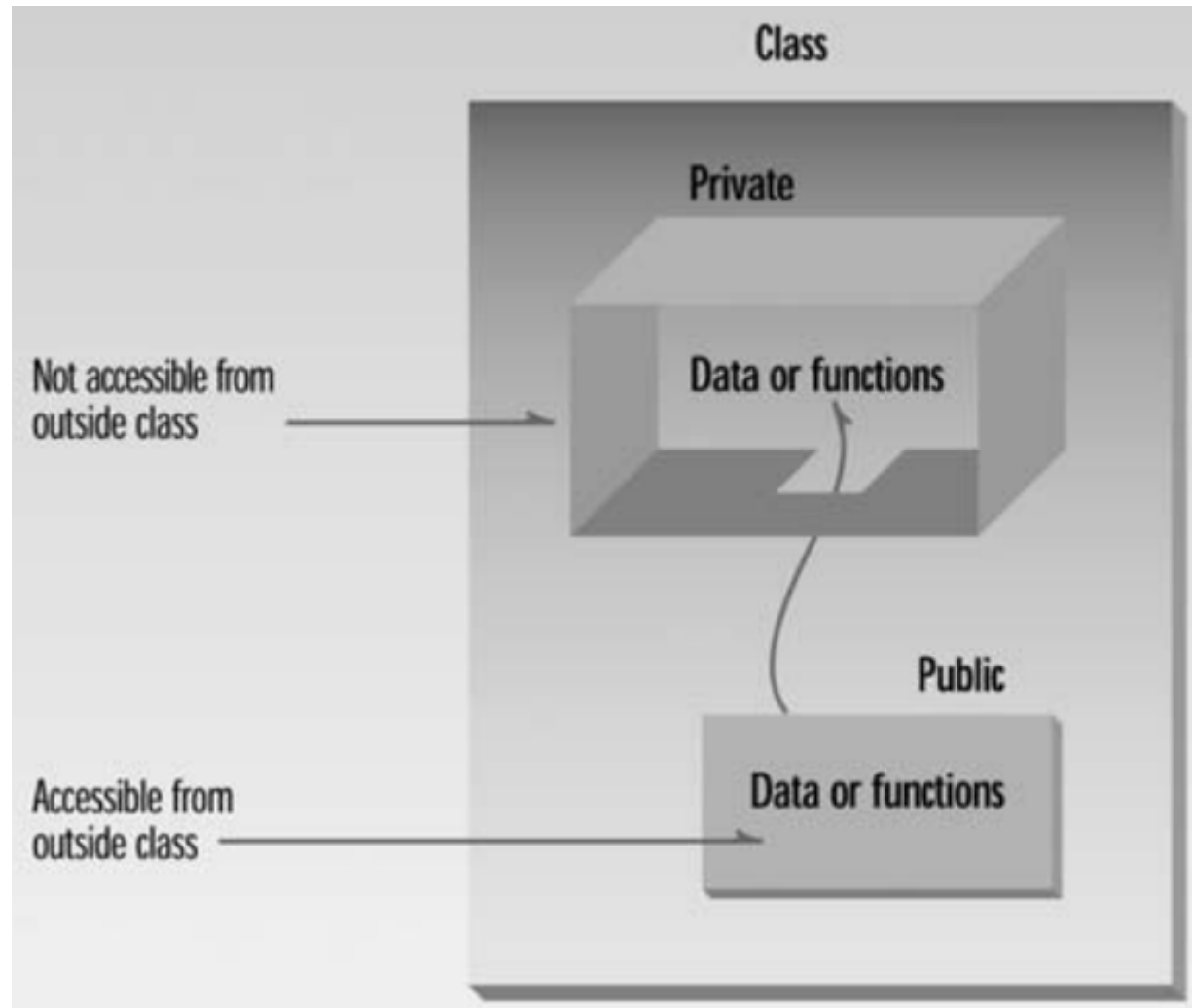
- The data items within a class are called **data members** (or sometimes member data).
- There can be any number of data members in a class, just as there can be any number of data items in a structure.
- **Member functions** are functions that are included within a class.
- (In some object-oriented languages, such as Smalltalk, member functions are called methods; some writers use this term in C++ as well.)



Private and Public Class Members

- A key feature of object-oriented programming is **data hiding**.
- It means that data is concealed within a class so that it cannot be accessed by functions outside the class.
- The primary mechanism for hiding data is to put it in a class and make it private.
- Private data or functions can only be accessed from within the class.
- Public data or functions, on the other hand, are accessible from outside the class.

Private and Public Class Members



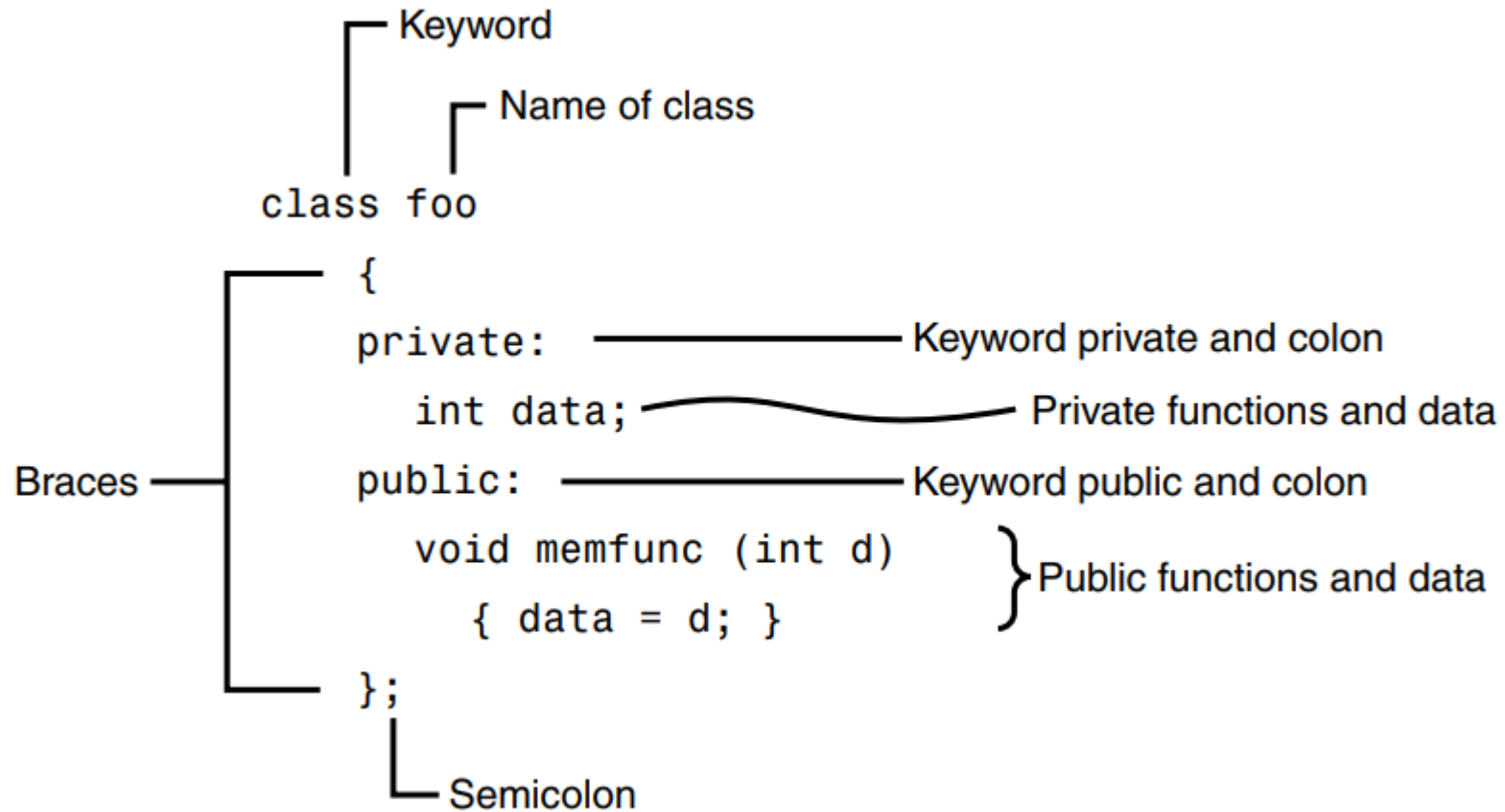


Private Data, Public Functions

- Usually the data within a class is private and the functions are public.
- This is a result of the way classes are used.
- The data is hidden so it will be safe from accidental manipulation,
- While the functions that operate on the data are public so they can be accessed from outside the class.
- However, there is no rule that says data must be private and functions public; in some circumstances you may find you'll need to use private functions and public data.



Syntax of a Class Definition





Member Functions

- The member functions in the `smallobj` class perform operations that are quite common in classes: **setting and retrieving the data** stored in the class.
- The `setdata()` function accepts a value as a parameter and sets the `somedata` variable to this value.
- The `showdata()` function displays the value stored in `somedata`.

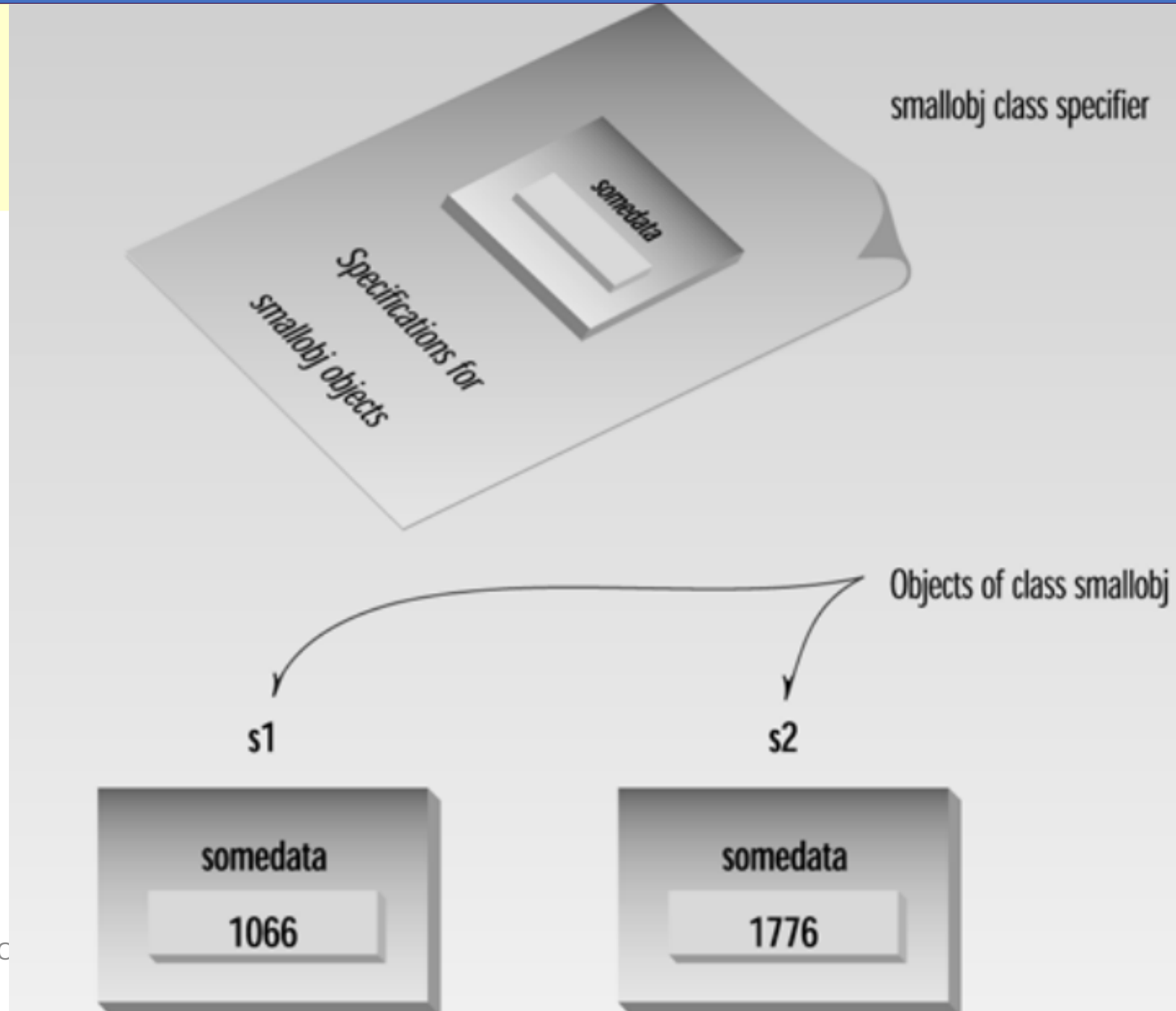


Defining Objects

- The first statement in `main()` `smallobj s1, s2;` defines two objects, `s1` and `s2`, of class `smallobj`.
- Defining an object is similar to defining a variable of any data type: Space is set aside for it in memory.
- Defining objects in this way means creating them. This is also called **instantiating** them, because an instance of the class is created.
- An object is an **instance** (that is, a specific example) **of a class**.
- Objects are sometimes called **instance variables**.

Instantiating an Object

- Some object-oriented languages refer to calls to member functions as messages.
- Thus the call `s1.showdata()`; can be thought of as sending a message to `s1` telling it to show its data.





Another Example

```
class part //define class
{
    private:
        int modelnumber; //ID number of widget
        int partnumber; //ID number of widget part
        float cost; //cost of part
    public:
        void setpart(int mn, int pn, float c) //set data
        {
            modelnumber = mn;
            partnumber = pn;
            cost = c;
        }

        void showpart() //display data
        {
            cout << "Model " << modelnumber;
            cout << ", part " << partnumber;
            cout << ", costs $" << cost << endl;
        }
};
```



Another Example

```
int main()
{
    part part1; //define object
    // of class part
    part1.setpart(6244, 373, 217.55F); //call member function
    part1.showpart(); //call member function
    return 0;
}
```



Constructors



Constructors

- A constructor is a function that is used to initialize an object, once it is created.
- A constructor is a member function that is executed automatically whenever an object is created.
- The term constructor is sometimes abbreviated `ctor`, especially in comments in program listings.



Constructors

- Note that the name of the constructor is that of the class itself.
- Constructors do not have a return type.
- There are several ways to initialize variables using a constructor.
- One way is to use a ":" after the constructor's name, followed by the name of each variable and then its initial value in parenthesis.

```
class Counter
{
    private:
        unsigned int count;
    public:
        Counter() : count(0)
```

Constructors: Initializer Lists

- If multiple members must be initialized, they're separated by commas. The result is the initializer list (sometimes called by other names, such as the member-initialization list).

```
someClass() : m1(7), m2(33), m2(4) ←———— initializer list
{ }
```

- Still another way to initialize values in a constructor is by assigning the values in the body of the constructor.



Constructors: Initialization

- Another way to initialize variables in the constructor is by assignment.
- Have a look at the following example:

```
class circle {  
    private:  
        double radius, area, circum;  
    public:  
        circle(double r)  
        {    radius = r; }  
  
        void calc_circumference() {  
            circum = 2 * 3.1416 * radius;  
        }  
  
        void calc_area() {  
            area = 3.1416 * radius * radius;  
        }  
}
```

Constructors: Initialization

```
void print_circle() {  
    calc_circumference();  
    calc_area();  
    cout << "Circle: Radius = " << radius <<  
    ", Circumference = " << circum << ", Area = "  
    << area << endl;  
}  
};  
  
int main(void) {  
    circle c1(4.0), c2(5.0), c3(1.0);  
  
    c1.print_circle();  
    c2.print_circle();  
    c3.print_circle();  
}
```



Summary

- This session was a first look at Object Oriented Programming.
- A class can be considered as an extension of a structure – contains both data and functions on that data.
- Access Modifiers such as public and private control access to both data and functions.



Summary

- In C++, dynamically allocated memory must be explicitly deallocated in order to avoid memory leaks
- An associated concept to memory leaks is that of a dangling pointer, which is a pointer that refers to a variable or a data element that has been deleted and deallocated.
- Function overloading is the concept of assigning the same functionality to a function using different parameter types, order or number.
- Overloaded functions must have the same name.